Intégration de Modèles IA

dans les Systèmes Logiciels

Lab 2 : Entraîner et Sérialiser un Modèle

Dr. El Hadji Bassirou TOURE
École Supérieure Polytechnique
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

L2 GLSI / DUT-INFO2 — Semestre 4 — 2026

Résumé

Ce lab vous guide dans la construction de la première brique concrète de SénSanté : le **modèle ML**. Vous chargerez le dataset de 500 patients, séparerez les données en entraînement et test, entraînerez un RandomForestClassifier, évaluerez ses performances (accuracy + matrice de confusion), puis sérialiserez le modèle en fichier `model.pkl`. À la fin, le dossier `models/` ne sera plus vide et le tag passera à `v1`.

⊙ Ce que vous allez apprendre

À l'issue de ce lab, vous saurez :

- Charger un dataset CSV avec pandas et préparer les features
- Séparer les données en ensembles d'entraînement et de test
- Entraîner un modèle RandomForest avec scikit-learn
- Évaluer un modèle : accuracy, matrice de confusion, rapport de classification
- Sérialiser un modèle avec `joblib.dump()` et le recharger
- Utiliser le modèle sérialisé pour prédire sur de nouvelles données

Durée estimée : 2 heures
Prérequis : Lab 1 terminé (dépôt `sensante` sur GitHub, tag `v0`, `patients_dakar.csv` dans `data/`)
Tag Git : `v1` | **Compétence :** C3 (Sérialiser un modèle ML)

Table des matières

1 Introduction : du dataset au fichier modèle **2**

2 Étape 1 : Préparer l'environnement **2**

3 Étape 2 : Charger et préparer les données **3**

4 Étape 3 : Séparer entraînement et test **4**

5 Étape 4 : Entraîner le modèle **6**

6 Étape 5 : Évaluer le modèle **7**

7 Étape 6 : Sérialiser le modèle **9**

8 Étape 7 : Tester le modèle sérialisé **11**

9 Étape 8 : Git — sauvegarder la version v1 **12**

10 Résumé **15**

11 Exercices **15**

12 Conclusion **16**

1 Introduction : du dataset au fichier modèle

En Lab 1, vous avez mis en place le projet : Git, Python, structure, dataset. Vous avez exploré les données de 500 patients de la région de Dakar. Aminata, étudiante en L2, regarde son dossier `models/` et constate qu'il est **vide**. « On a les données, mais pas de modèle. Comment on en fabrique un ? »

En CM2, on a vu qu'un modèle ML est un programme qui *apprend* à partir de données. En CM3, on a vu l'architecture complète : le modèle est un **fichier** qui sera chargé par l'API FastAPI. Ce lab construit exactement ce fichier.

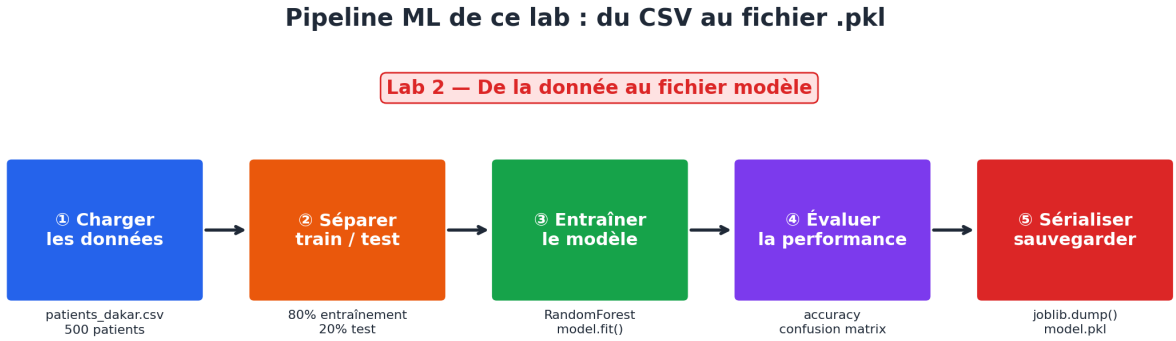

FIGURE 1 – Le pipeline ML de ce lab. En 5 étapes, on passe du fichier CSV au fichier `model.pkl`.

⋆ Intuition

Analogie : apprendre à cuisiner
Imaginez que vous apprenez à reconnaître les plats sénégalais. On vous montre 400 photos avec le nom du plat (c'est l'**entraînement**). Puis on vous teste avec 100 nouvelles photos sans le nom (c'est le **test**).
C'est exactement ce que fait le modèle ML : il *apprend* les patterns dans 400 patients, puis on vérifie s'il diagnostique correctement les 100 patients restants.

2 Étape 1 : Préparer l'environnement

☒ À faire

Étape 1.1 : Activer l'environnement virtuel et installer les dépendances

```
# Se placer dans le projet
cd sensante

# Activer l'environnement virtuel
# Windows :
.venv\Scripts\activate
# macOS / Linux :
source .venv/bin/activate
```

☒ **À faire**

Étape 1.2 : Installer scikit-learn et joblib

```
pip install scikit-learn joblib matplotlib seaborn
```

☒ **À faire**

Étape 1.3 : Mettre à jour `requirements.txt`

```
pip freeze > requirements.txt
```

☒ **À faire**

Étape 1.4 : Créer le fichier de travail

Créez le fichier `notebooks/train_model.py` dans VS Code. C'est dans ce fichier que vous écrirez tout le code de ce lab.

3 Étape 2 : Charger et préparer les données

☒ **À faire**

Étape 2.1 : Charger le dataset

```python
import pandas as pd
import numpy as np

# Charger le dataset
df = pd.read_csv("data/patients_dakar.csv")

# Verifier les dimensions
print(f"Dataset : {df.shape[0]} patients, {df.shape[1]} colonnes")
print(f"\nColonnes : {list(df.columns)}")
print(f"\nDiagnostics :\n{df['diagnostic'].value_counts()}")
```

```
Dataset : 500 patients, 9 colonnes

Colonnes : ['age', 'sexe', 'temperature', 'tension_sys',
            'toux', 'fatigue', 'maux_tete', 'region', 'diagnostic']

Diagnostics :
palu       135
grippe     130
sain       125
typh       110
Name: diagnostic, dtype: int64
```

☐ Explication pédagogique

Que contient le dataset ?
Chaque ligne est un patient. Les colonnes sont :

- **Features** (entrées du modèle) : `age`, `sexe`, `temperature`, `tension_sys`, `toux`, `fatigue`, `maux_tete`, `region`
- **Target** (ce qu'on veut prédire) : `diagnostic` — 4 classes : paludisme, grippe, typhoïde, sain

Le modèle va apprendre la relation entre les features et le diagnostic. C'est un problème de **classification supervisée** (CM2).

☒ À faire

Étape 2.2 : Préparer les features et la cible

```
from sklearn.preprocessing import LabelEncoder

# Encoder les variables categoriques en nombres
# Le modele ne comprend que des nombres !
le_sexe = LabelEncoder()
le_region = LabelEncoder()

df['sexe_encoded'] = le_sexe.fit_transform(df['sexe'])
df['region_encoded'] = le_region.fit_transform(df['region'])

# Definir les features (X) et la cible (y)
feature_cols = ['age', 'sexe_encoded', 'temperature', 'tension_sys',
                'toux', 'fatigue', 'maux_tete', 'region_encoded']

X = df[feature_cols]
y = df['diagnostic']

print(f"Features : {X.shape}")   # (500, 8)
print(f"Cible : {y.shape}")      # (500,)
```

△ Point d'attention

Pourquoi encoder ?
Le modèle ML ne comprend que les **nombres**. Si `sexe` vaut `"M"` ou `"F"`, le modèle ne sait pas quoi en faire. Le `LabelEncoder` transforme `"F"` → 0 et `"M"` → 1. De même pour les régions. C'est une étape obligatoire avant tout entraînement. En CM2, on a dit que le modèle travaille avec des *features numériques* — c'est ici que ça se concrétise.

4 Étape 3 : Séparer entraînement et test

⋆ Intuition

Analogie : réviser vs passer l'examen
Si vous révisez un cours et que l'examen contient exactement les mêmes questions que vos fiches de révision, votre note ne prouve rien — vous avez juste **mémorisé**.
Un vrai examen teste avec des questions *nouvelles*. C'est pareil pour le modèle : on l'entraîne

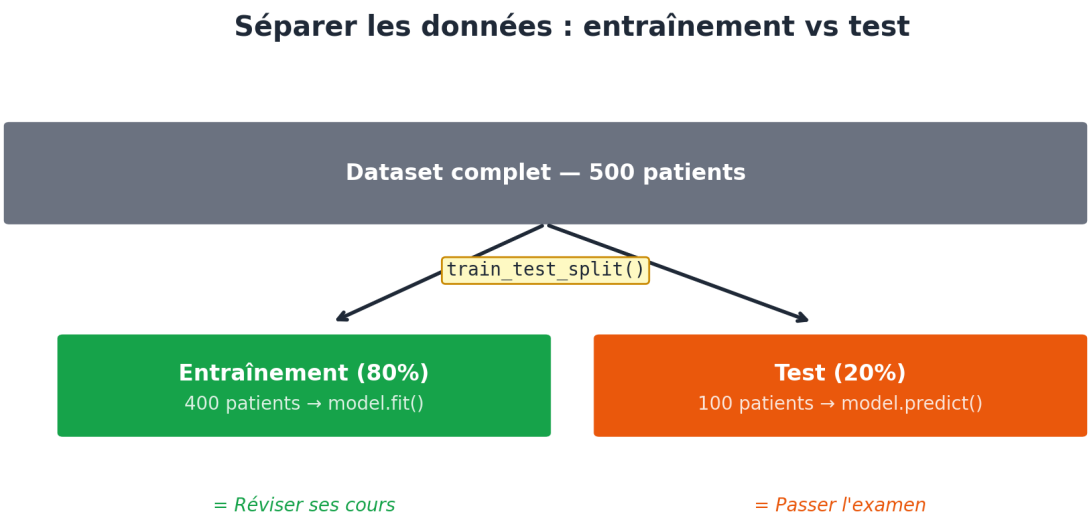

FIGURE 2 – La séparation train/test. 80% des données pour apprendre, 20% pour vérifier. Comme réviser ses cours puis passer l'examen.

sur 80% des données, et on le teste sur les 20% restants qu'il n'a **jamais vus**.

☒ À faire

Étape 3.1 : Séparer les données

```
from sklearn.model_selection import train_test_split

# 80% pour l'entrainement, 20% pour le test
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,        # 20% pour le test
    random_state=42,      # Pour avoir les memes resultats a chaque fois
    stratify=y            # Garder les memes proportions de diagnostics
)

print(f"Entrainement : {X_train.shape[0]} patients")
print(f"Test : {X_test.shape[0]} patients")
```

```
Entrainement : 400 patients
Test : 100 patients
```

☐ Explication pédagogique

Pourquoi `random_state=42` ?
`random_state` fixe le « hasard » de la séparation. Si vous et votre camarade utilisez le même `random_state`, vous aurez exactement les mêmes données dans train et test. C'est essentiel pour la **reproductibilité** — un concept clé en science.

42 est une convention (c'est la réponse à la grande question dans *Le Guide du voyageur galactique*). Vous pouvez utiliser n'importe quel entier.

☐ Explication pédagogique

Pourquoi `stratify=y` ?
Sans `stratify`, la séparation aléatoire pourrait mettre 0 patients *typhoïde* dans le test. Avec `stratify=y`, chaque classe est représentée proportionnellement dans train et test. Si le dataset contient 27% de paludisme, le test en contiendra aussi environ 27%.

5 Étape 4 : Entraîner le modèle

◇ Concept fondamental

RandomForestClassifier — en une phrase
Un **Random Forest** est un ensemble de nombreux arbres de décision. Chaque arbre vote pour un diagnostic, et le diagnostic final est celui qui reçoit le plus de votes. C'est le **scrutin majoritaire** appliqué au Machine Learning.
Vous n'avez pas besoin de comprendre les maths derrière. Il suffit de savoir que c'est un algorithme **fiable**, **simple à utiliser**, et qui donne de bons résultats sur des données tabulaires comme les nôtres.

☒ À faire

Étape 4.1 : Entraîner le modèle (5 lignes !)

```python
from sklearn.ensemble import RandomForestClassifier

# Creer le modele
model = RandomForestClassifier(
    n_estimators=100,      # 100 arbres de decision
    random_state=42        # Reproductibilite
)

# Entrainer sur les donnees d'entrainement
model.fit(X_train, y_train)

print("Modele entraine !")
print(f"Nombre d'arbres : {model.n_estimators}")
print(f"Nombre de features : {model.n_features_in_}")
print(f"Classes : {list(model.classes_)}")
```

```
Modele entraine !
Nombre d'arbres : 100
Nombre de features : 8
Classes : ['grippe', 'palu', 'sain', 'typh']
```

• Point clé à retenir

C'est fait. En 5 lignes de code, vous avez entraîné un modèle de Machine Learning. `model.fit()` est la ligne la plus importante : c'est là que le modèle *apprend* les patterns dans les données. À ce stade, le modèle existe **en mémoire** (dans la variable `model`). Si vous fermez Python, il disparaît. L'étape 6 (sérialisation) le sauvera sur le disque.

6 Étape 5 : Évaluer le modèle

⋆ Intuition

On ne juge pas un étudiant sur les exercices de révision.
On le juge sur l'examen — des questions qu'il n'a pas vues. De même, on évalue le modèle sur les données de **test** (les 100 patients qu'il n'a jamais vus pendant l'entraînement).

☒ À faire

Étape 5.1 : Prédire sur les données de test

```
# Predire sur les donnees de test
y_pred = model.predict(X_test)

# Comparer les 10 premieres predictions avec la realite
comparison = pd.DataFrame({
    'Vrai diagnostic': y_test.values[:10],
    'Prediction': y_pred[:10]
})
print(comparison)
```

```
  Vrai diagnostic Prediction
0          grippe     grippe
1            palu       palu
2            sain       sain
3            typh       typh
4            palu       palu
5          grippe     grippe
6            sain       sain
7            typh     grippe
8            palu       palu
9            sain       sain
```

☒ À faire

Étape 5.2 : Calculer l'accuracy

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy : {accuracy:.2%}")
```

```
Accuracy : 89.00%
```

☐ Explication pédagogique

Que signifie 89% d'accuracy ?
Sur 100 patients de test, le modèle a donné le bon diagnostic pour **89 d'entre eux**. Il s'est trompé pour 11 patients.
C'est un bon score pour un premier modèle avec des données simples. En production, on voudrait mieux — mais pour ce cours d'intégration, c'est largement suffisant. L'objectif est d'apprendre à **intégrer** le modèle, pas à optimiser ses performances.

☒ À faire

Étape 5.3 : Matrice de confusion

```python
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns

# Matrice de confusion
cm = confusion_matrix(y_test, y_pred, labels=model.classes_)
print("Matrice de confusion :")
print(cm)

# Rapport de classification
print("\nRapport de classification :")
print(classification_report(y_test, y_pred))
```

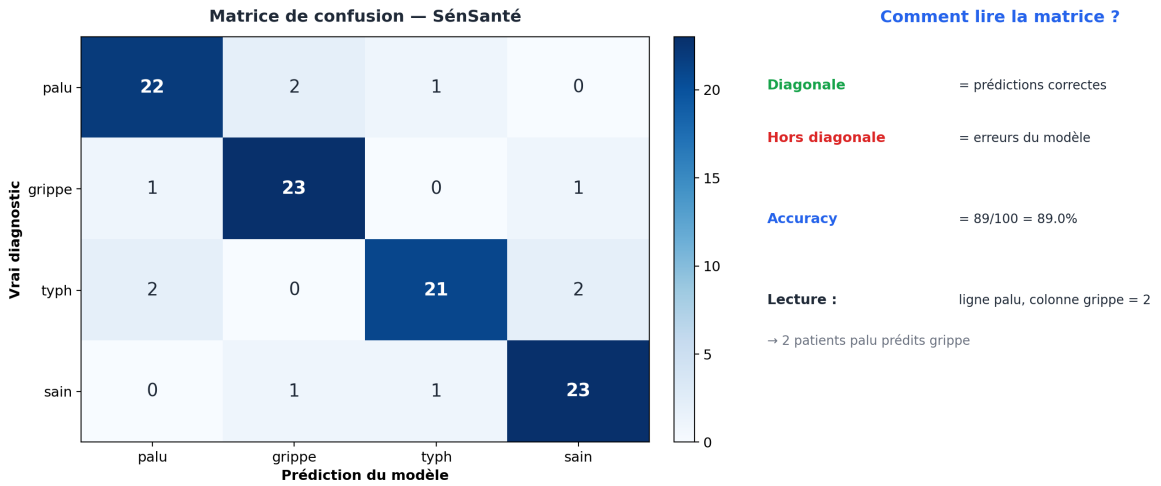

FIGURE 3 – Matrice de confusion de SénSanté. La diagonale montre les prédictions correctes. Accuracy globale : 89%.

☐ Explication pédagogique

Comment lire la matrice de confusion ?
— Chaque **ligne** = le vrai diagnostic

— Chaque **colonne** = la prédiction du modèle
— Les nombres sur la **diagonale** = prédictions correctes
— Les nombres **hors diagonale** = erreurs

Exemple : si la case (palu, grippe) = 2, cela signifie que 2 patients qui avaient réellement le paludisme ont été diagnostiqués grippe par le modèle.

☒ **À faire**

Étape 5.4 : (Optionnel) Visualiser la matrice de confusion

```
# Visualiser avec seaborn
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=model.classes_,
            yticklabels=model.classes_)
plt.xlabel('Prediction du modele')
plt.ylabel('Vrai diagnostic')
plt.title('Matrice de confusion - SenSante')
plt.tight_layout()
plt.savefig('figures/confusion_matrix.png', dpi=150)
plt.show()

print("Figure sauvegardee dans figures/confusion_matrix.png")
```

7 Étape 6 : Sérialiser le modèle

C'est l'**étape clé** de ce lab. Tout ce qui précède (charger, séparer, entraîner, évaluer) est le travail du Data Scientist. La sérialisation est le **pont** vers le monde du développeur.

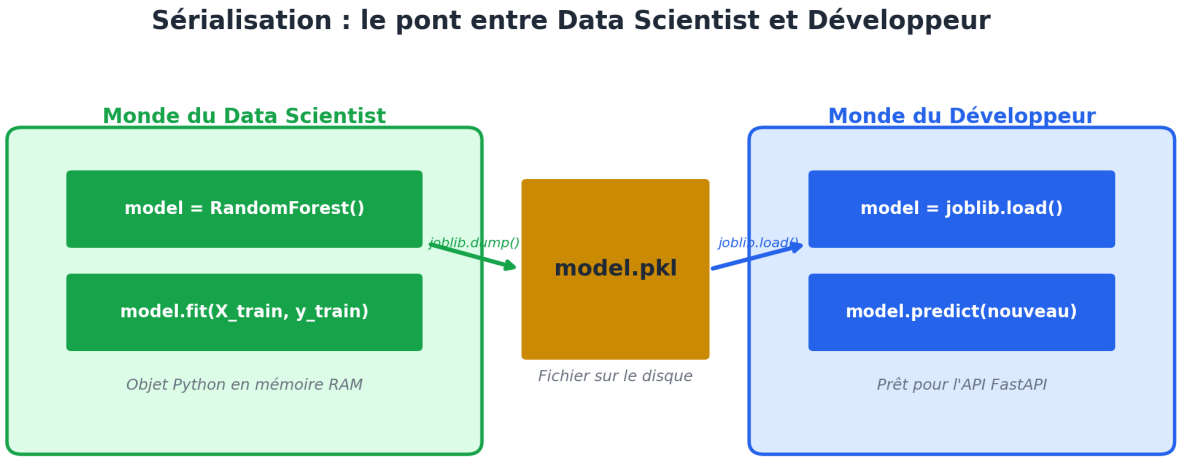

FIGURE 4 – La sérialisation transforme l'objet Python en mémoire en un fichier `.pkl` sur le disque. Ce fichier sera chargé par l'API FastAPI en Lab 3.

◇ Concept fondamental

Sérialisation — en une phrase
La **sérialisation** est l'opération qui transforme un objet en mémoire (le modèle entraîné) en un **fichier** sur le disque. L'opération inverse (charger le fichier pour recréer l'objet) s'appelle la **désérialisation**.
C'est comme prendre un plat cuisiné et le mettre **sous vide** pour le conserver. Quand on veut le manger, on le réchauffe (désérialise) et il est prêt.

☒ À faire

Étape 6.1 : Sauvegarder le modèle

```
import joblib
import os

# Creer le dossier models/ s'il n'existe pas
os.makedirs("models", exist_ok=True)

# Serialiser le modele
joblib.dump(model, "models/model.pkl")

# Verifier la taille du fichier
size = os.path.getsize("models/model.pkl")
print(f"Modele sauvegarde : models/model.pkl")
print(f"Taille : {size / 1024:.1f} Ko")
```

```
Modele sauvegarde : models/model.pkl
Taille : 842.3 Ko
```

• Point clé à retenir

Le modèle est un fichier !
Le fichier `models/model.pkl` contient tout ce que le modèle a appris. Il peut être copié, envoyé par email, stocké sur GitHub, chargé par un autre programme. C'est la **base de l'intégration** : à partir de maintenant, n'importe quel développeur peut charger ce fichier et faire des prédictions — sans avoir besoin de refaire l'entraînement.

☒ À faire

Étape 6.2 : Sauvegarder aussi les encodeurs

```
# Sauvegarder les encodeurs (indispensables pour les nouvelles donnees)
joblib.dump(le_sexe, "models/encoder_sexe.pkl")
joblib.dump(le_region, "models/encoder_region.pkl")

# Sauvegarder la liste des features (pour reference)
joblib.dump(feature_cols, "models/feature_cols.pkl")

print("Encodeurs et metadata sauvegardes.")
```

△ Point d'attention

Pourquoi sauvegarder les encodeurs ?
Le modèle a été entraîné avec `sexe_encoded` (0 = F, 1 = M). Si en Lab 3, l'API reçoit `"sexe": "F"`, elle doit savoir que F = 0. C'est le `LabelEncoder` qui contient cette correspondance.
Si vous ne sauvegardez pas les encodeurs, l'API ne pourra pas transformer les données d'entrée correctement. Le modèle recevra des valeurs incohérentes et donnera des résultats absurdes.

8 Étape 7 : Tester le modèle sérialisé

☒ À faire

Étape 7.1 : Recharger le modèle depuis le fichier

```
# Simuler ce que fera l'API en Lab 3 :
# Charger le modele DEPUIS LE FICHIER (pas depuis la memoire)
model_loaded = joblib.load("models/model.pkl")
le_sexe_loaded = joblib.load("models/encoder_sexe.pkl")
le_region_loaded = joblib.load("models/encoder_region.pkl")

print(f"Modele recharge : {type(model_loaded).__name__}")
print(f"Classes : {list(model_loaded.classes_)}")
```

```
Modele recharge : RandomForestClassifier
Classes : ['grippe', 'palu', 'sain', 'typh']
```

☒ À faire

Étape 7.2 : Prédire pour un nouveau patient (simulation)

```
# Un nouveau patient arrive au centre de sante de Medina
nouveau_patient = {
    'age': 28,
    'sexe': 'F',
    'temperature': 39.5,
    'tension_sys': 110,
    'toux': True,
    'fatigue': True,
    'maux_tete': True,
    'region': 'Dakar'
}

# Encoder les valeurs categoriques
sexe_enc = le_sexe_loaded.transform([nouveau_patient['sexe']])[0]
region_enc = le_region_loaded.transform([nouveau_patient['region']])[0]

# Preparer le vecteur de features
features = [
    nouveau_patient['age'],
    sexe_enc,
    nouveau_patient['temperature'],
```

```
    nouveau_patient['tension_sys'],
    int(nouveau_patient['toux']),
    int(nouveau_patient['fatigue']),
    int(nouveau_patient['maux_tete']),
    region_enc
]

# Predire
diagnostic = model_loaded.predict([features])[0]
probas = model_loaded.predict_proba([features])[0]
proba_max = probas.max()

print(f"\n--- Resultat du pre-diagnostic ---")
print(f"Patient : {nouveau_patient['sexe']}, {nouveau_patient['age']} ans
    ")
print(f"Diagnostic : {diagnostic}")
print(f"Probabilite : {proba_max:.1%}")
print(f"\nProbabilites par classe :")
for classe, proba in zip(model_loaded.classes_, probas):
    bar = '#' * int(proba * 30)
    print(f"  {classe:8s} : {proba:.1%} {bar}")
```

```
--- Resultat du pre-diagnostic ---
Patient : F, 28 ans
Diagnostic : palu
Probabilite : 72.3%

Probabilites par classe :
  grippe   : 15.2% ####
  palu     : 72.3% ######################
  sain     :  3.1% #
  typh     :  9.4% ##
```

• Point clé à retenir

Vous venez de simuler exactement ce que fera l'API en Lab 3.
Le processus est :

1. Recevoir les données d'un patient (ici un dictionnaire, en Lab 3 un JSON)
2. Encoder les catégories (sexe, région)
3. Construire le vecteur de features
4. Appeler `model.predict()` et `model.predict_proba()`
5. Retourner le diagnostic et la probabilité

La seule différence en Lab 3 : les données arriveront via HTTP au lieu d'être écrites dans le code.

9 Étape 8 : Git — sauvegarder la version v1

☒ À faire

Étape 8.1 : Vérifier l'état du dépôt

```
git status
```

```
On branch main
Changes not staged for commit:
  modified:   requirements.txt

Untracked files:
  models/model.pkl
  models/encoder_sexe.pkl
  models/encoder_region.pkl
  models/feature_cols.pkl
  notebooks/train_model.py
```

☒ À faire

Étape 8.2 : Ajouter au `.gitignore` les fichiers temporaires

Ouvrez `.gitignore` et ajoutez :

```
# Fichiers temporaires matplotlib
*.png
!figures/*.png
```

☒ À faire

Étape 8.3 : Commit et tag

```
# Ajouter tous les fichiers modifies et nouveaux
git add .

# Committer avec un message descriptif
git commit -m "Lab 2 : modele RandomForest entraine et serialise (v1)"

# Envoyer sur GitHub
git push origin main

# Creer le tag v1
git tag v1

# Envoyer le tag
git push origin v1
```

```
[main a3b7c21] Lab 2 : modele RandomForest entraine et serialise (v1)
 6 files changed, 142 insertions(+)
 create mode 100644 models/model.pkl
 create mode 100644 models/encoder_sexe.pkl
 create mode 100644 models/encoder_region.pkl
 create mode 100644 models/feature_cols.pkl
 create mode 100644 notebooks/train_model.py
To https://github.com/moussa-diallo/sensante.git
 * [new tag]         v1 -> v1
```

• Point clé à retenir

Version v1 marquée ! Votre projet a maintenant un modèle entraîné et sérialisé. L'historique Git ressemble à :

```
git log --oneline
a3b7c21 Lab 2 : modele RandomForest (v1)
f1e2d3c Lab 1 : structure projet (v0)
```

10 Résumé

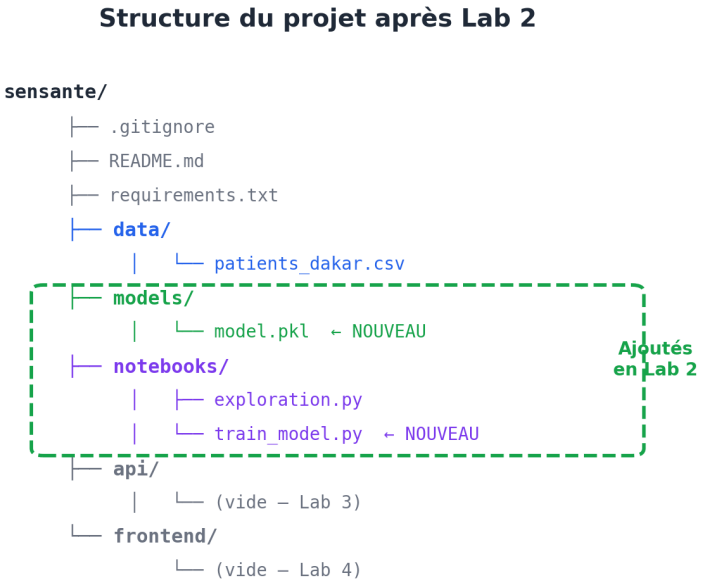

FIGURE 5 – Structure du projet après Lab 2. Les fichiers dans `models/` et `notebooks/train_model.py` sont nouveaux.

• Point clé à retenir

Ce que vous avez accompli dans ce lab :

#	Action	Statut
1	Installer scikit-learn, joblib, matplotlib, seaborn	✓
2	Charger `patients_dakar.csv` (500 patients)	✓
3	Encoder les variables catégoriques (sexe, région)	✓
4	Séparer train (400) / test (100) avec stratification	✓
5	Entraîner un RandomForestClassifier (100 arbres)	✓
6	Évaluer : accuracy ≈ 89%, matrice de confusion	✓
7	Sérialiser : `model.pkl` + encodeurs + feature_cols	✓
8	Tester le modèle rechargé sur un nouveau patient	✓
9	Git : `commit` + `push` + `tag v1`	✓

11 Exercices

☒ À faire

Exercice 1 — Importance des features : Ajoutez le code suivant à `train_model.py` pour afficher les features les plus importantes pour le modèle :

```
importances = model.feature_importances_
for name, imp in sorted(zip(feature_cols, importances),
                        key=lambda x: x[1], reverse=True):
```

```
    print(f"  {name:20s} : {imp:.3f}")
```

Quelle feature est la plus importante ? Cela vous semble-t-il logique médicalement ?
Exercice 2 — Tester avec d'autres patients : Créez 3 patients fictifs avec des profils différents (un jeune sans symptômes, un adulte avec forte fièvre, un patient âgé avec toux). Prédisez leur diagnostic avec le modèle sérialisé. Les résultats sont-ils cohérents ?
Exercice 3 — Réflexion : Le modèle a 89% d'accuracy. Dans un contexte médical réel, est-ce suffisant ? Quels seraient les risques d'un faux diagnostic ? Discutez en 5 lignes.

12 Conclusion

⊙ Ce que vous allez apprendre

Félicitations ! Vous avez :

- ✓ Entraîné un modèle RandomForest sur le dataset SénSanté
- ✓ Évalué ses performances (89% d'accuracy)
- ✓ Sérialisé le modèle en `models/model.pkl`
- ✓ Vérifié que le modèle rechargé fonctionne sur un nouveau patient
- ✓ Créé le tag `v1` sur GitHub

Le dossier `models/` n'est plus vide. Vous avez le **fichier modèle** dont parle le CM3 — la brique centrale de l'architecture Model-as-a-Service.
En Lab 3 : Vous construirez l'**API FastAPI** qui charge `model.pkl` au démarrage et expose un endpoint `POST /predict`. Le code de l'étape 7 de ce lab deviendra une *route FastAPI*. L'agent de santé à Tambacounda pourra envoyer les symptômes d'un patient via HTTP et recevoir un diagnostic en retour. Tag : `v2`.

Références

[1] C. Huyen, *Designing Machine Learning Systems*, O'Reilly, 2022. Chapitre 7 (Model Deployment and Prediction Service).

[2] Documentation scikit-learn : https://scikit-learn.org/stable/

[3] Documentation joblib : https://joblib.readthedocs.io/en/latest/

[4] F. Pedregosa et al., "Scikit-learn : Machine Learning in Python", *JMLR*, 2011.